CMPS 450: November 10, 2025

Subprograms with Recursion

Dr. Maida

Programs with stack-dynamic local variables

1. Stack-dynamic local variables are the local variables in procedures that we use all the time. Languages with stack-dynamic local variables support recursion. Sebesta calls them stack dynamic b/c their storage is dynamically allocated using a new AR on the stack each time the procedure is called.

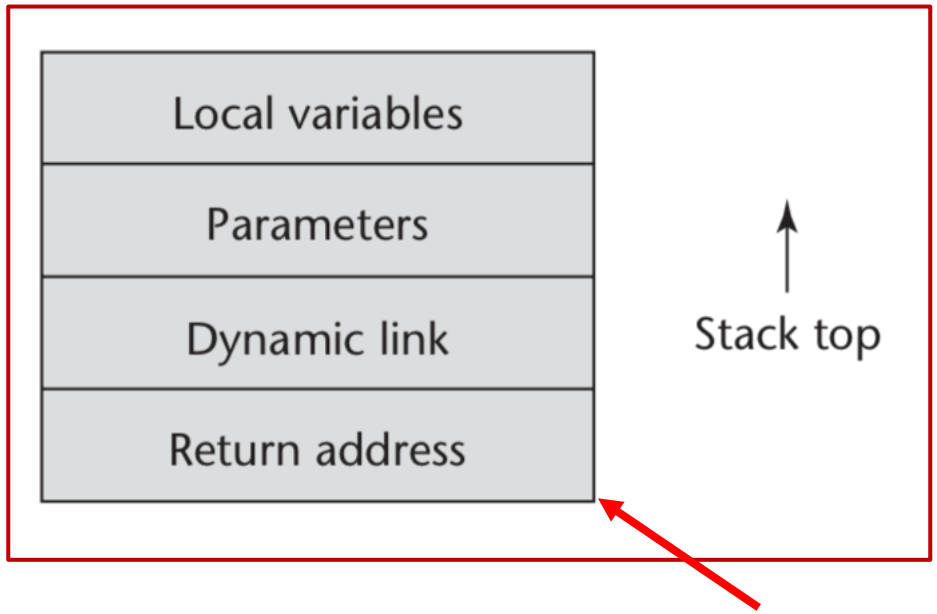
2. Recall that the fields of an AR occupy consecutive regions of memory. That is, they are just blocks of memory.

3. Now that we will put more than one AR on a stack, we must be able point to the beginning of an AR. Specifically, an AR on the top of the stack needs a pointer to the beginning of the AR below it on the stack, and so on until we reach the bottom of the stack.

4. This will be called the dynamic link.

An AR for programs with stack-dynamic local variables

Figure 10.3



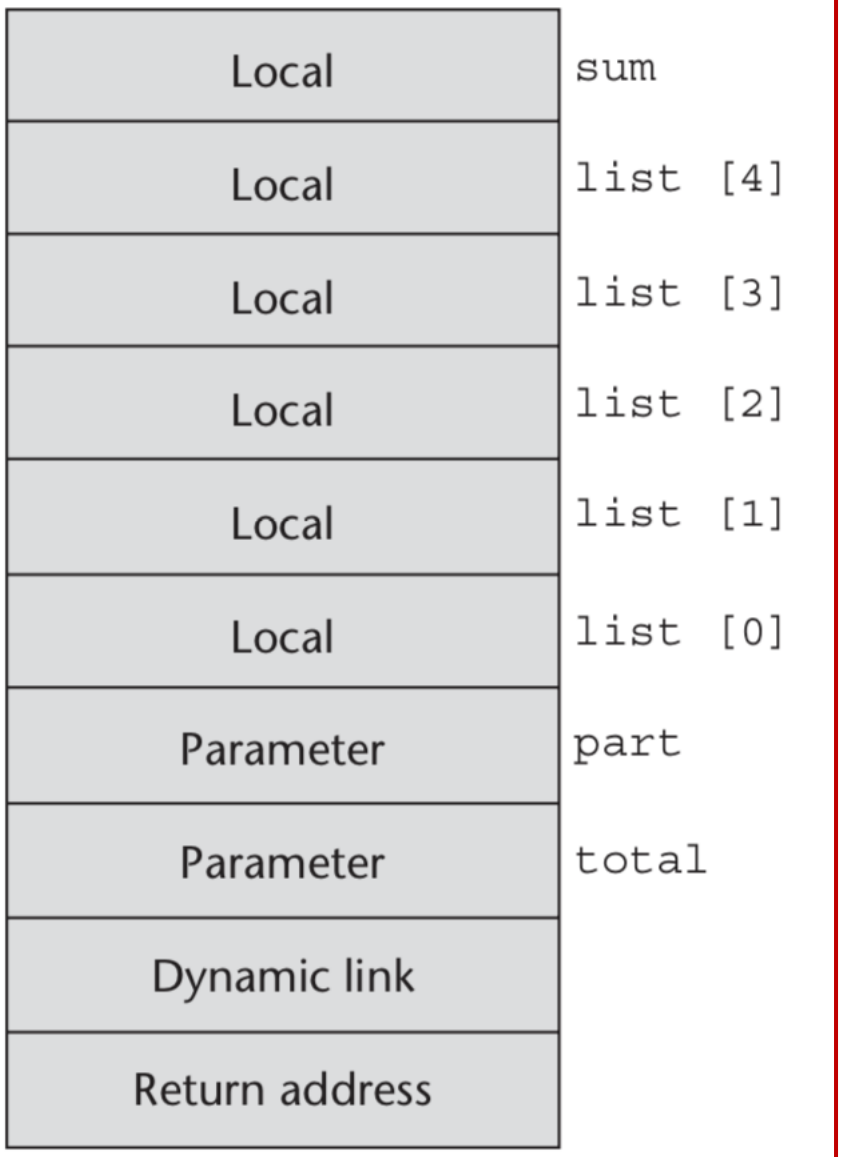
1. This AR is the same as the one for simple subprograms, except that it has one more field to hold the dynamic link.

2. Since these ARs are put on a stack, the arrow shows the direction in which the stack grows.

The next slide shows how to build a particular activation record for a specific procedure.

Figure 10.4

How to build an AR




```
void sub(float total, int part) {
   int list[5];
   float sum;
   . . .
}
```

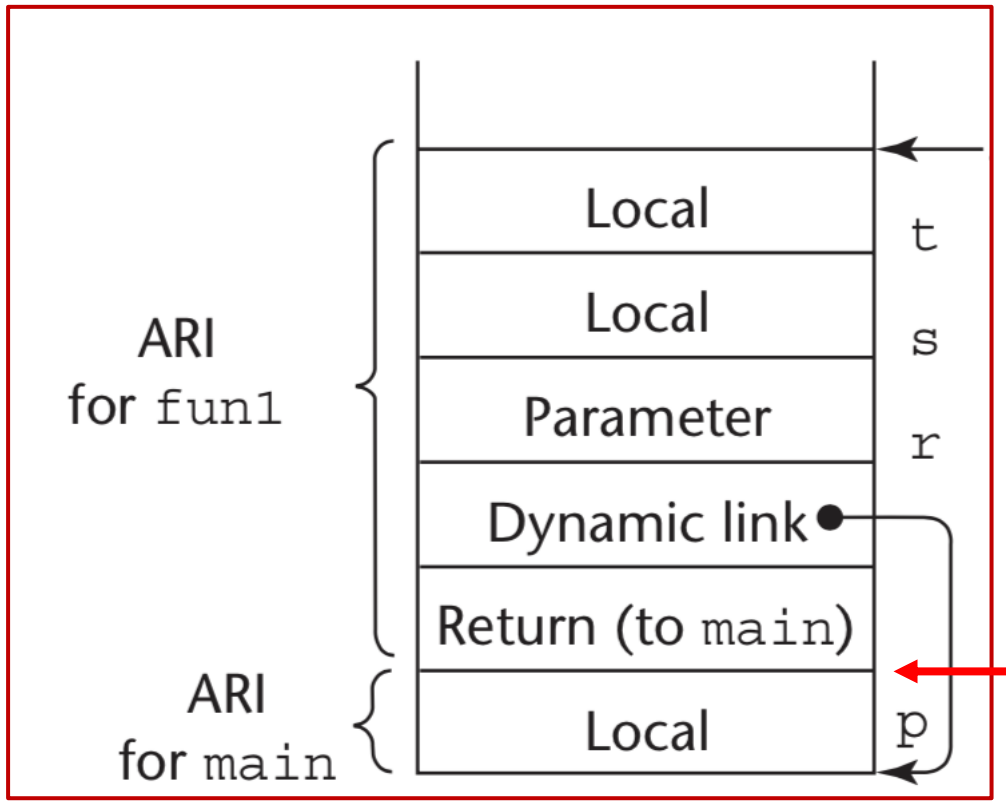
1. Notice that the array “list” has five elements.

2. Notice that the variables are listed in the AR in the order that they are declared in the procedure.

3. Remember that the bottom of the AR is the front.

A stack with two activation records

Figure 10.5 (left)

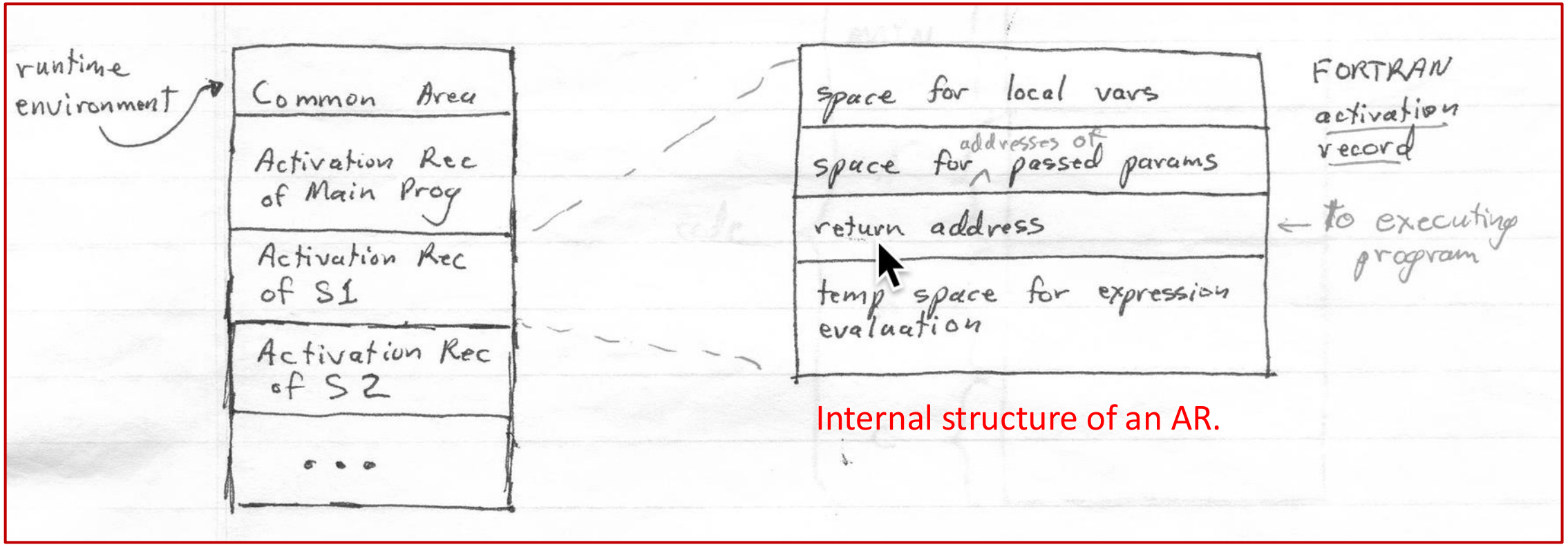


1. main() has called fun1(). fun1() takes one parameter (named r) and has two local variables (named s and t).

2. The size of an AR depends on the number of declarations in the procedure or function.

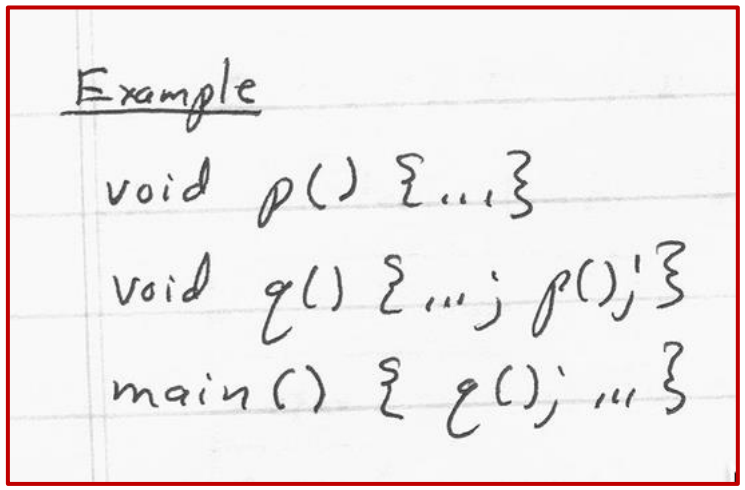
The environment pointer points to the beginning of the last AR on the stack. That is, the beginning of the AR at the stack top. The dynamic link points to the beginning of the AR just below it on the stack.

Original Fortran Memory Layout: No Recursion Allowed



In Fortran, there was only a static area. Activation records (ARs) were stored in static area with 1 AR per procedure. Worked if you didn't do recursion. With recursion, you need one AR per procedure invocation. Recursion needs a stack mechanism which pushes ARs on a stack at runtime and pops when procedure returns.

Recap



1. Another example w/o recursion. Therefore, main(), q(), and p() could have their activation records in the static area of the program.

2. All modern languages use a runtime stack.

3. Let's see more detail.

The basic idea is to put ARs on a runtime stack when a procedure is invoked during execution. Then pop the AR when the procedure returns. Internally, we use three kinds of pointers to keep track of these ARs. These are:

1. Environment pointer (ep)
2. Control (dynamic) link to keep track of procedure calls.
3. Access (static) link to keep track of nesting levels in the source code.

What these pointers let you do

1. If you just have an environment pointer and a control link, you can implement dynamic scoping.
2. However, we have seen that dynamic scoping is obsolete.
3. If you want lexical scoping, you need the third pointer type, the access link.

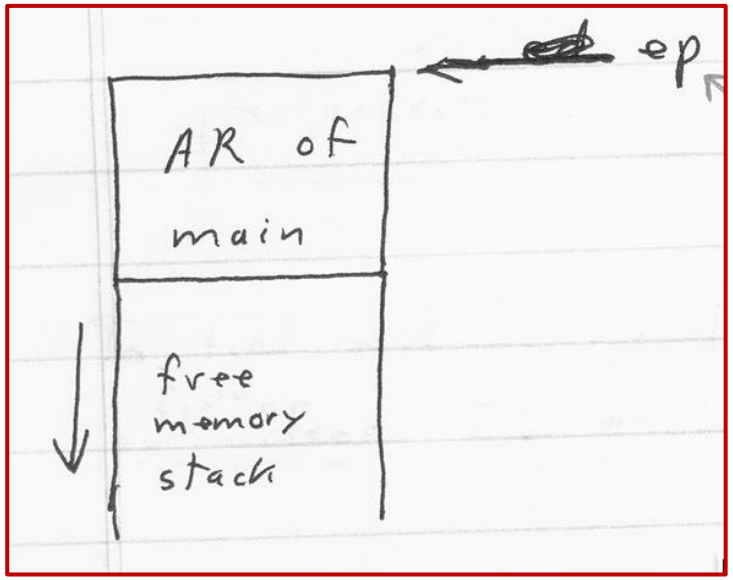
First, let's learn how the environment pointers and control links work. The we'll come back to the access link.

Environment Pointer and Control Link

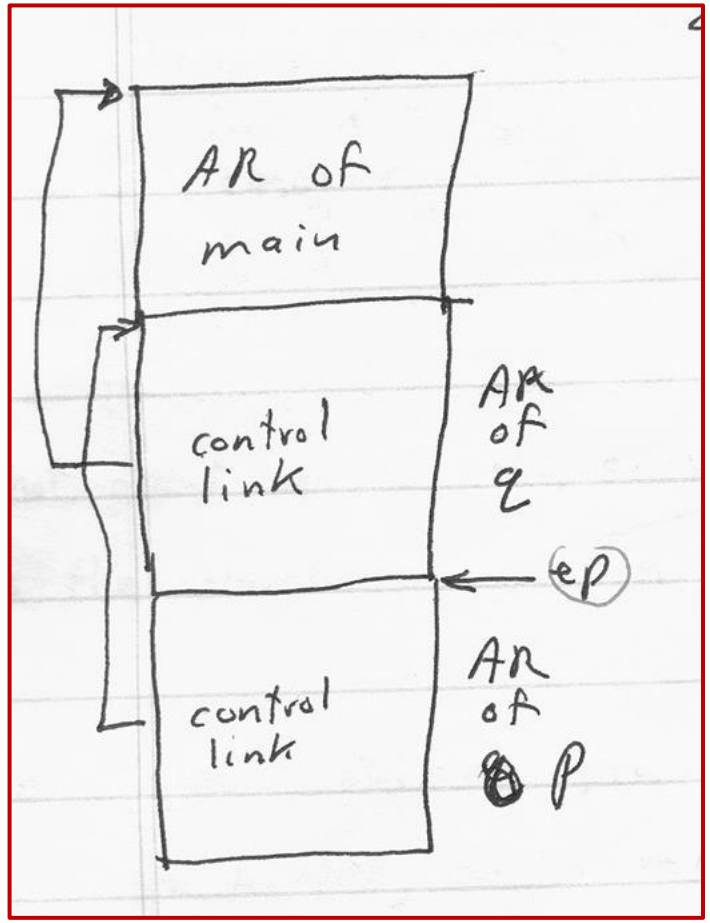
Example

```
void p() {...}
void q() {...; p();}
main() { q(); ...}
```

Before program starts:



After p() is called.



1. Environment pointer points to AR for currently executing procedure.
2. Control (dynamic) link points to AR on stack for calling procedure to resume execution after callee returns.
3. This support dynamic scoping but not static. For this, also need track static nesting levels.

Storage allocation for classes in Java: use of static keyword

```
class Body {
    long idNum;
    String name = "<unnamed>";
    Body orbits = null;

    static long nextID = 0;

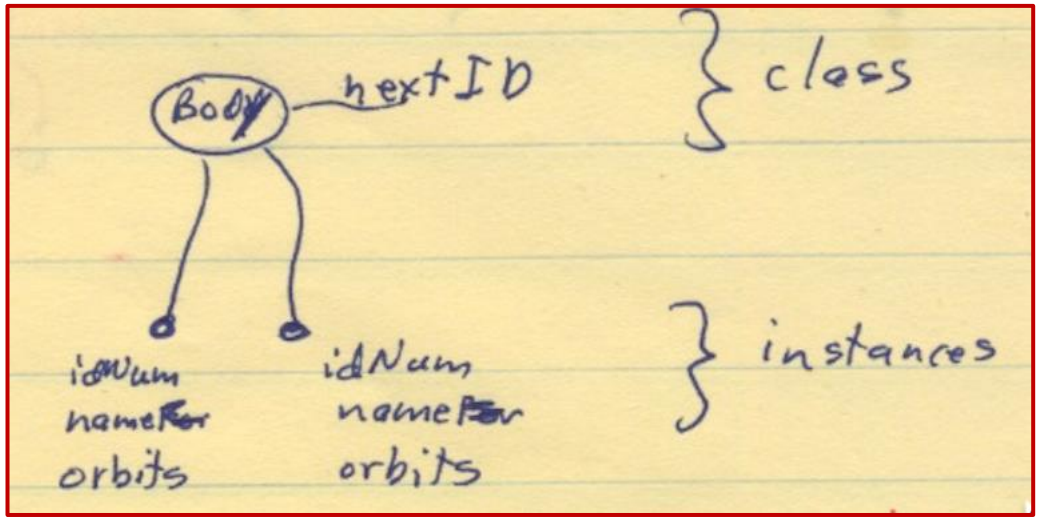
    Body () { idNum = nextID++; }}

public static void main() {
    Body earth = new Body();
    Body sun = new Body();
}
```

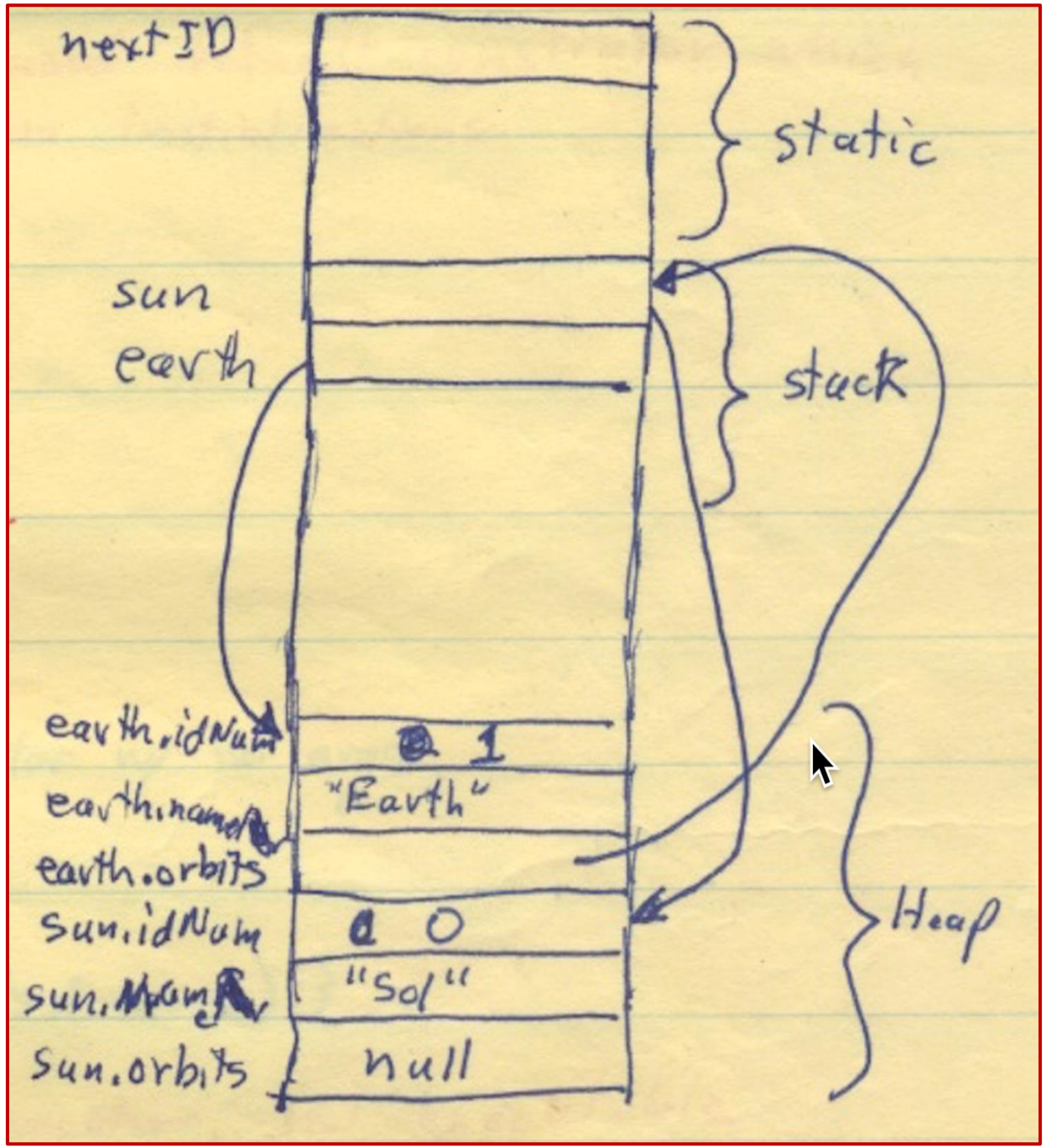
Allocated in static area of memory. Shared among all class instances.

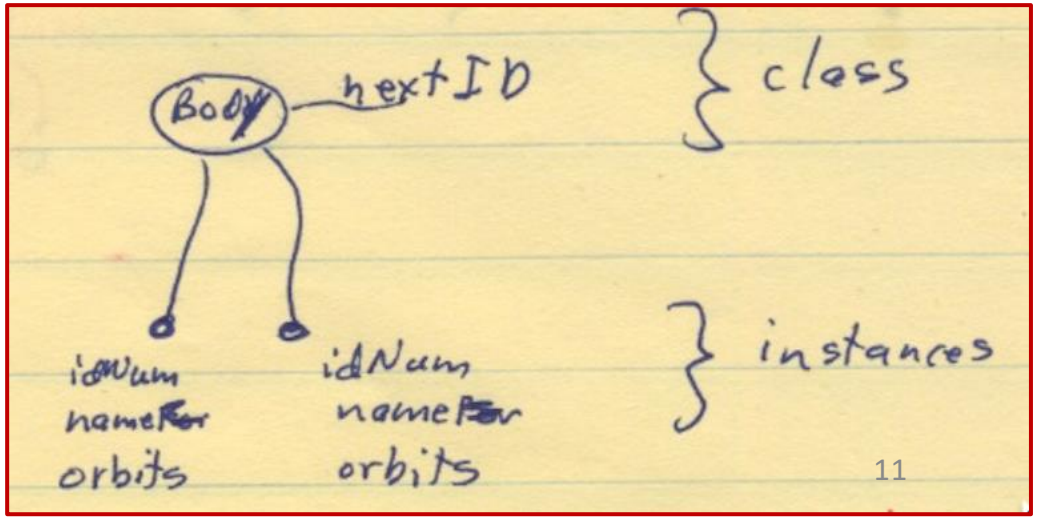
References for “earth” and ”sun” are stored on the stack.

Allocated on heap.



Revisiting Memory Layout





Class instance management in C++ versus Java

1. C++ uses destructors. Whenever instance declaration goes out of scope, a destructor is called. The destructor is defined by the programmer. If the programmer forgets to free storage in the destructor, there is a memory leak. If the programmer inadvertently frees storage that is still used there is a “use after free issue.” (Potential address out of bounds, core dump).

2. Java uses garbage collection and the above issues do not arise. Java is much safer.

3. Trade-off btw speed and safety. Garbage collection offers safety but incurs runtime overhead.

Review: What does unsafe mean?

Semantics of Assignment Statements

Consider this assignment statement.

```
x = x + 1;
```

1. The meaning of the two occurrences of “x” is different because of their different contexts.

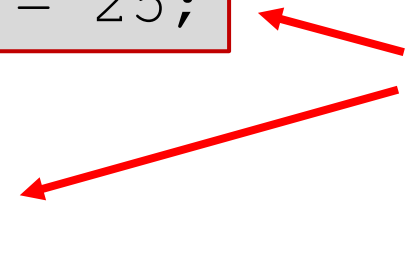
2. The “x” on the LHS refers to a memory location (l-value).

3. The “x” on the RHS refers to the contents of that location (r-value).

Cryptic Error Message: Not an l-value

```
x + 1 = 25;
```

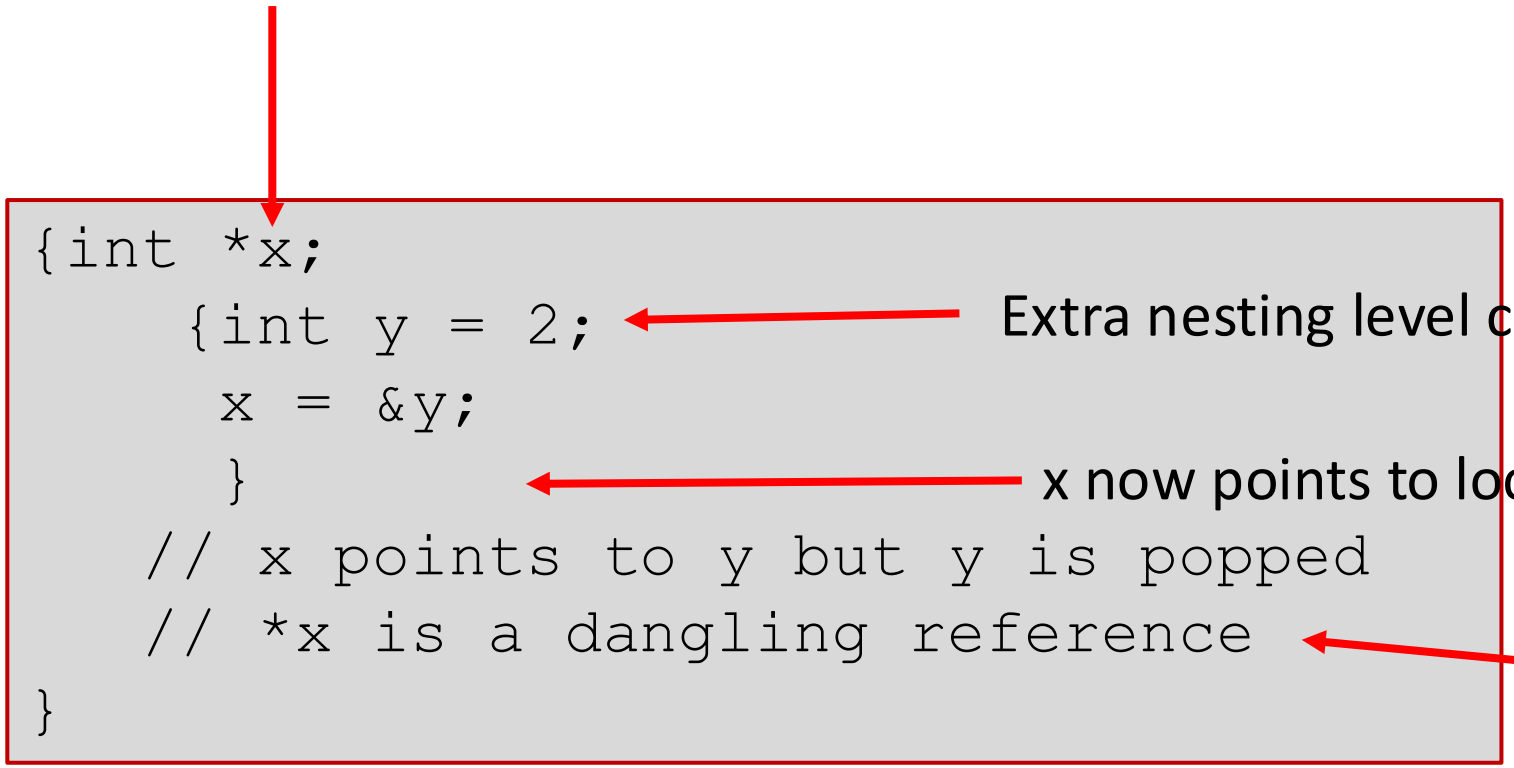
```
1 = 25;
```



1. These are illegal.
2. If you have ever seen “not an l-value error” from your compiler, this is the kind of situation that it is complaining about.
3. Only variables are allowed on the LHS of an assignment statement because they can refer to memory locations.

Understanding the stack helps w/ understanding some pointer issues in C

Declare “x” as a pointer to an integer.



Another C example w/ the stack

Creates a dangling pointer, which points to data that is no longer valid.

1. Calling procedure dangle() puts an AR on the stack that holds the local variable x.

2. Upon return, the address of int is returned but the stack is popped, so that memory location is either unused or used for something else.

```
int * dangle() {
     int x;
     return &x;
}
\\ points to memory location on stack
\\ after returning, points to memory
\\ location no longer used.
```

Make an educated guess

```
void main() {
    {int x;
     printf("%d\n", x);
     x = 1;}
    {int y;
     printf("%d\n", y);}
}
```



Because y is not initialized, the output of this program is not defined.

Based on your knowledge of how activation records are pushed and popped from the stack, make a guess on what the output of this problem might be.

Functional Programming Paradigm

1. In functional programming, functions are considered as 1st-class objects. By this we mean that functions can be manipulated as if they were data objects in the program.

2. For example, they can be the value of a variable.

3. If you put the variable in the function call position of an expression, the function will be applied.

4. A few slides later, we show an example from a toy functional programming language.

5. It creates two functions w/ the identical definitions. However, the functions have free variables that take on values from their environments. Since the functions are defined in different environments, they return different values, even when called with the same arguments.

Warm Up: Scheme Expression for Beginners

Scheme expression:

```
(let ((x 1)
      (y 2))
    (+ x y))
```

Initialize two variables, no type declarations, same mechanism as Python.

Evaluate expression and return. No explicit return statement.

C/C++/Java translation:

```
{int x = 1;
 int y = 2;
 return x+y;}
```

The let in scheme performs a similar function as the curly braces in C.

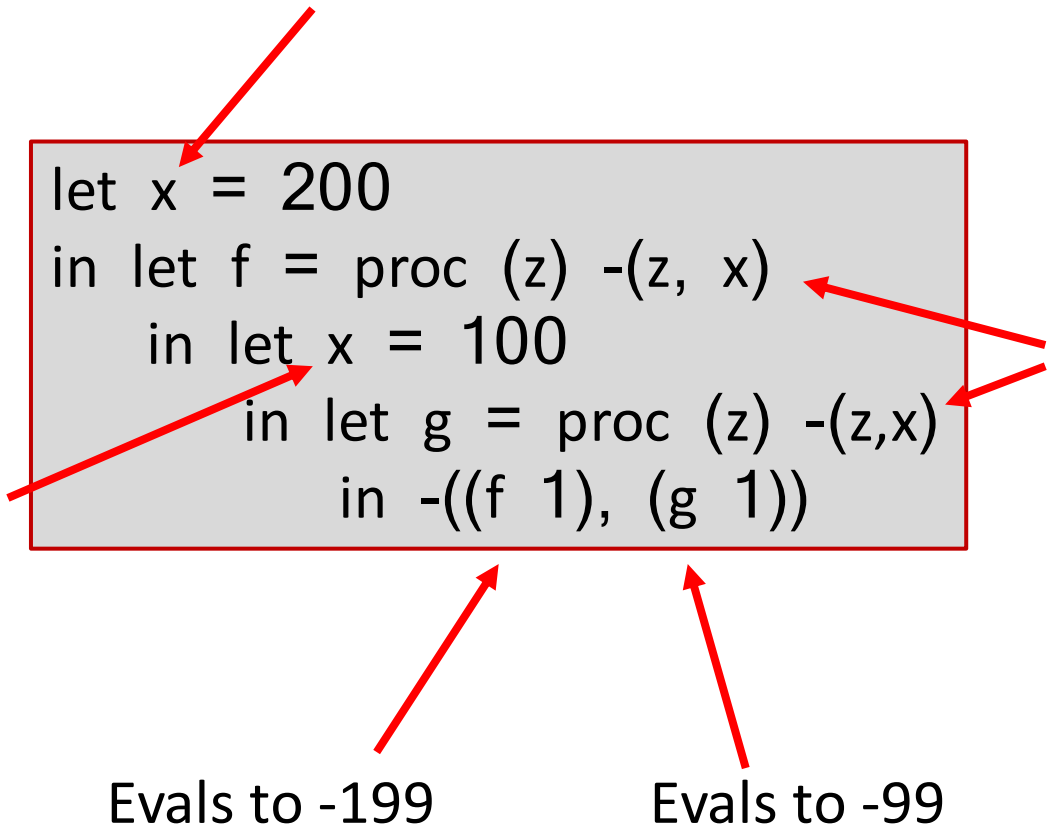
Example Scheme Program

Remember: Each let is similar to a nested curly brace.

```
(let ((x 200))
    (let ((f (lambda (z) (- z x))))
        (let ((x 100))
            (let ((g (lambda (z) (- z x))))
                (- (f 1) (g 1)))))))
```

1. This code has two anonymous (lambda) functions w/ exactly the same definition.
2. An anonymous function does not have a name but it can be the value of a variable.
3. Each function has a declared formal parameter z used in the body of the function.
4. Also, each function has a free variable x, whose value comes from the enclosing environment.
5. The functions are used in the last line of the program.
6. This expression returns the value: -100.

Lexical Scoping in a Functional Programming Language



How to implement? Beyond the scope of this class.

1. A scheme-like functional language with a more familiar syntax.
2. Functions are 1st-class objects and bound to values variables.
3. Functions bound to f and g have free variables, so the underlying implementation must keep track of the value of x at the time the function was defined. (closure)
4. B/c of lexical scoping the 2nd declaration of x shadows the 1st declaration.
5. Both f and g have same definition and same args, but defined in different environments, and return different values!

The next slides address the issue of memory safety

1. It provides proof-of-concept of how something like a buffer-overflow bug can be used to trick a program into executing arbitrary code and create a malicious program.

2. C is highly vulnerable to this and C++ is less vulnerable if you use modern constructs like smart pointers, introduced in C++11.

3. Modern programming languages like Rust and Swift make this impossible, and so are more secure. Python, Java, and Go are also secure.

4. Companies have huge C/C++ code bases which are expensive to translate. Furthermore, they are still writing new C/C++ code.

Security Issue: C Language Example

Arg count

A C-string is an array of chars.
So, **argv is an array of strings.

```
int main(int argc, char **argv) {
  unsigned long a[1];
  a[3] = 0x7ffff7b36cebUL;
  return 0;
}
```

Declares an array of length 1.

Set out-of-bounds value and makes program behavior undefined. Similar to a buffer overflow. Actual platform specific behavior is to change return address of main.

Platform specific error and then crash:

```
undef: Error: .netrc file is readable by others.
undef: Remove password or make file unreadable by others.
```

From: *Why Rust?* by Jim Blandy, 2015, p. 3, O'Reilly.

Explanation of previous slide

The machine code the C compiler generated for my `main` function happens to place the array `a` on the stack three words before the return address, so storing `0x7ffff7b36cebUL` in `a[3]` changes poor `main`'s return address to point into the midst of code in the C standard library that consults one's `.netrc` file for a password. When my `main` returns, execution resumes not in `main`'s caller, but at the machine code for these lines from the library:

```
warnx(_("Error: .netrc file is readable by others."));
warnx(_("Remove password or make file unreadable by others."));
                goto bad;
```

Quote from Jim Blandy, p. 3.

.netrc stores information for FTP.